Lab 2: Cats vs Dogs

In this lab, you will train a convolutional neural network to classify an image into one of two classes: "cat" or "dog". The code for the neural networks you train will be written for you, and you are not (yet!) expected to understand all provided code. However, by the end of the lab, you should be able to:

1. Understand at a high level the training loop for a machine learning model.
2. Understand the distinction between training, validation, and test data.
3. The concepts of overfitting and underfitting.
4. Investigate how different hyperparameters, such as learning rate and batch size, affect the success of training.
5. Compare an ANN (aka Multi-Layer Perceptron) with a CNN.

What to submit

Submit a PDF file containing all your code, outputs, and write-up from parts 1-5. You can produce a PDF of your Google Colab file by going to **File > Print** and then save as PDF. The Colab instructions has more information.

Do not submit any other files produced by your code.

Include a link to your colab file in your submission.

Please use Google Colab to complete this assignment. If you want to use Jupyter Notebook, please complete the assignment and upload your Jupyter Notebook file to Google Colab for submission.

With Colab, you can export a PDF file using the menu option `File -> Print` and save as PDF file. **Adjust the scaling to ensure that the text is not cutoff at the margins.**

Colab Link

Include a link to your colab file here

Colab Link: https://drive.google.com/file/d/12wHDcjM3xftvLjc8tqC2ylA8vlxC1nbU/view?usp=drive_link

In [4]:
```
import numpy as np
import time
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import torchvision
from torch.utils.data.sampler import SubsetRandomSampler
import torchvision.transforms as transforms
```

Part 0. Helper Functions

We will be making use of the following helper functions. You will be asked to look at and possibly modify some of these, but you are not expected to understand all of them.

You should look at the function names and read the docstrings. If you are curious, come back and explore the code *after* making some progress on the lab.

In [5]:
```
##############################################################################
# Data Loading

def get_relevant_indices(dataset, classes, target_classes):
    """ Return the indices for datapoints in the dataset that belongs to the
    desired target classes, a subset of all possible classes.

    Args:
        dataset: Dataset object
        classes: A list of strings denoting the name of each class
        target_classes: A list of strings denoting the name of desired classes
                        Should be a subset of the 'classes'
    Returns:
        indices: list of indices that have labels corresponding to one of the
                 target classes
    """
    indices = []
    for i in range(len(dataset)):
```

```
        # Check if the label is in the target classes
        label_index = dataset[i][1] # ex: 3
        label_class = classes[label_index] # ex: 'cat'
        if label_class in target_classes:
            indices.append(i)
    return indices

def get_data_loader(target_classes, batch_size):
    """ Loads images of cats and dogs, splits the data into training, validation
    and testing datasets. Returns data loaders for the three preprocessed datasets.

    Args:
        target_classes: A list of strings denoting the name of the desired
                        classes. Should be a subset of the argument 'classes'
        batch_size: A int representing the number of samples per batch

    Returns:
        train_loader: iterable training dataset organized according to batch size
        val_loader: iterable validation dataset organized according to batch size
        test_loader: iterable testing dataset organized according to batch size
        classes: A list of strings denoting the name of each class
    """

    classes = ('plane', 'car', 'bird', 'cat',
               'deer', 'dog', 'frog', 'horse', 'ship', 'truck')
    ########################################################################
    # The output of torchvision datasets are PILImage images of range [0, 1].
    # We transform them to Tensors of normalized range [-1, 1].
    transform = transforms.Compose(
        [transforms.ToTensor(),
         transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))])
    # Load CIFAR10 training data
    trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
                                            download=True, transform=transform)
    # Get the list of indices to sample from
    relevant_indices = get_relevant_indices(trainset, classes, target_classes)

    # Split into train and validation
    np.random.seed(1000) # Fixed numpy random seed for reproducible shuffling
    np.random.shuffle(relevant_indices)
    split = int(len(relevant_indices) * 0.8) #split at 80%

    # split into training and validation indices
    relevant_train_indices, relevant_val_indices = relevant_indices[:split], relevant_indices[split:]
    train_sampler = SubsetRandomSampler(relevant_train_indices)
    train_loader = torch.utils.data.DataLoader(trainset, batch_size=batch_size,
                                               num_workers=1, sampler=train_sampler)
    val_sampler = SubsetRandomSampler(relevant_val_indices)
    val_loader = torch.utils.data.DataLoader(trainset, batch_size=batch_size,
                                             num_workers=1, sampler=val_sampler)
    # Load CIFAR10 testing data
    testset = torchvision.datasets.CIFAR10(root='./data', train=False,
                                           download=True, transform=transform)
    # Get the list of indices to sample from
    relevant_test_indices = get_relevant_indices(testset, classes, target_classes)
    test_sampler = SubsetRandomSampler(relevant_test_indices)
    test_loader = torch.utils.data.DataLoader(testset, batch_size=batch_size,
                                              num_workers=1, sampler=test_sampler)
    return train_loader, val_loader, test_loader, classes

#############################################################################
# Training
def get_model_name(name, batch_size, learning_rate, epoch):
    """ Generate a name for the model consisting of all the hyperparameter values

    Args:
        config: Configuration object containing the hyperparameters
    Returns:
        path: A string with the hyperparameter name and value concatenated
    """
    path = "model_{0}_bs{1}_lr{2}_epoch{3}".format(name,
                                                   batch_size,
                                                   learning_rate,
                                                   epoch)
    return path

# Normalizing the labels cat and dog to binary 0/1.
# The equation used is norm_labels = (labels - min_val)/(max_val - min_val)
def normalize_label(labels):
    """
    Given a tensor containing 2 possible values, normalize this to 0/1
```

```
    Args:
        labels: a 1D tensor containing two possible scalar values
    Returns:
        A tensor normalize to 0/1 value
    """
    max_val = torch.max(labels)
    min_val = torch.min(labels)
    norm_labels = (labels - min_val)/(max_val - min_val)
    return norm_labels

def evaluate(net, loader, criterion):
    """ Evaluate the network on the validation set.

     Args:
         net: PyTorch neural network object
         loader: PyTorch data loader for the validation set
         criterion: The loss function
     Returns:
         err: A scalar for the avg classification error over the validation set
         loss: A scalar for the average loss function over the validation set
     """
    total_loss = 0.0
    total_err = 0.0
    total_epoch = 0
    for i, data in enumerate(loader, 0):
        inputs, labels = data
        labels = normalize_label(labels)  # Convert labels to 0/1
        outputs = net(inputs)
        loss = criterion(outputs, labels.float())
        corr = (outputs > 0.0).squeeze().long() != labels
        total_err += int(corr.sum())
        total_loss += loss.item()
        total_epoch += len(labels)
    err = float(total_err) / total_epoch
    loss = float(total_loss) / (i + 1)
    return err, loss

##############################################################################
# Training Curve
def plot_training_curve(path):
    """ Plots the training curve for a model run, given the csv files
    containing the train/validation error/loss.

    Args:
        path: The base path of the csv files produced during training
    """
    import matplotlib.pyplot as plt
    train_err = np.loadtxt("{}_train_err.csv".format(path))
    val_err = np.loadtxt("{}_val_err.csv".format(path))
    train_loss = np.loadtxt("{}_train_loss.csv".format(path))
    val_loss = np.loadtxt("{}_val_loss.csv".format(path))
    plt.title("Train vs Validation Error")
    n = len(train_err) # number of epochs
    plt.plot(range(1,n+1), train_err, label="Train")
    plt.plot(range(1,n+1), val_err, label="Validation")
    plt.xlabel("Epoch")
    plt.ylabel("Error")
    plt.legend(loc='best')
    plt.show()
    plt.title("Train vs Validation Loss")
    plt.plot(range(1,n+1), train_loss, label="Train")
    plt.plot(range(1,n+1), val_loss, label="Validation")
    plt.xlabel("Epoch")
    plt.ylabel("Loss")
    plt.legend(loc='best')
    plt.show()
```

Part 1. Visualizing the Data [7 pt]

We will make use of some of the CIFAR-10 data set, which consists of colour images of size 32x32 pixels belonging to 10 categories. You can find out more about the dataset at https://www.cs.toronto.edu/~kriz/cifar.html

For this assignment, we will only be using the cat and dog categories. We have included code that automatically downloads the dataset the first time that the main script is run.

In [6]:
```
# This will download the CIFAR-10 dataset to a folder called "data"
# the first time you run this code.
train_loader, val_loader, test_loader, classes = get_data_loader(
```

```
    target_classes=["cat", "dog"],
    batch_size=1) # One image per batch
```

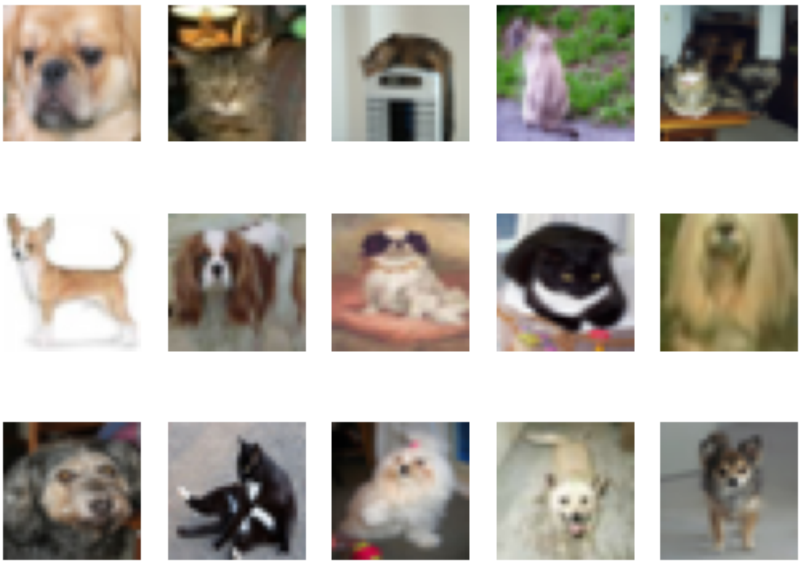
```
100%|██████████| 170M/170M [00:02<00:00, 64.8MB/s]
```

Part (a) -- 1 pt

Visualize some of the data by running the code below. Include the visualization in your writeup.

(You don't need to submit anything else.)

In [7]:
```
import matplotlib.pyplot as plt

k = 0
for images, labels in train_loader:
    # since batch_size = 1, there is only 1 image in `images`
    image = images[0]
    # place the colour channel at the end, instead of at the beginning
    img = np.transpose(image, [1,2,0])
    # normalize pixel intensity values to [0, 1]
    img = img / 2 + 0.5
    plt.subplot(3, 5, k+1)
    plt.axis('off')
    plt.imshow(img)

    k += 1
    if k > 14:
        break
```



Part (b) -- 3 pt

How many training examples do we have for the combined `cat` and `dog` classes? What about validation examples? What about test examples?

1. The number of training examples for cat and dog classes is 8000.
2. The number of validation examples for cat and dog classes is 2000.
3. The number of test examples for cat and dog classes is 2000.

In [10]:
```
# Number of Training examples
print(len(train_loader.sampler))

# Number of Validation examples
print(len(val_loader.sampler))

# Number of Test examples
print(len(test_loader.sampler))
```

```
8000
2000
2000
```

Part (c) -- 3pt

Why do we need a validation set when training our model? What happens if we judge the performance of our models using the training set loss/error instead of the validation set loss/error?

1. The reason why we use validation set to train our model is because it acts as a gauge for the test set during training. It allows us to tune hyperparameters. This ensures that the model learn from the training set and use the validation set to give us an honest estimate of how well the model handles new, unseen data.
2. If we only use training set loss/error, we would likely fall into the trap of Overfitting. NN is able to memorize the data points in training set to give a near 0% training error but fails to perform well on new, unseen data. Hence, we should completely train the model first using training data and validate it using unseen data in validation set to compute loss and error, avoiding the issue of Overfitting.

Part 2. Training [15 pt]

We define two neural networks, a `LargeNet` and `SmallNet`. We'll be training the networks in this section.

You won't understand fully what these networks are doing until the next few classes, and that's okay. For this assignment, please focus on learning how to train networks, and how hyperparameters affect training.

In [15]:
```
class LargeNet(nn.Module):
    def __init__(self):
        super(LargeNet, self).__init__()
        self.name = "large"
        self.conv1 = nn.Conv2d(3, 5, 5)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(5, 10, 5)
        self.fc1 = nn.Linear(10 * 5 * 5, 32)
        self.fc2 = nn.Linear(32, 1)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = x.view(-1, 10 * 5 * 5)
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        x = x.squeeze(1) # Flatten to [batch_size]
        return x
```

In [12]:
```
class SmallNet(nn.Module):
    def __init__(self):
        super(SmallNet, self).__init__()
        self.name = "small"
        self.conv = nn.Conv2d(3, 5, 3)
        self.pool = nn.MaxPool2d(2, 2)
        self.fc = nn.Linear(5 * 7 * 7, 1)

    def forward(self, x):
        x = self.pool(F.relu(self.conv(x)))
        x = self.pool(x)
        x = x.view(-1, 5 * 7 * 7)
        x = self.fc(x)
        x = x.squeeze(1) # Flatten to [batch_size]
        return x
```

In [13]:
```
small_net = SmallNet()
large_net = LargeNet()
```

Part (a) -- 2pt

The methods `small_net.parameters()` and `large_net.parameters()` produces an iterator of all the trainable parameters of the network. These parameters are torch tensors containing many scalar values.

We haven't learned how the parameters in these high-dimensional tensors will be used, but we should be able to count the number of parameters. Measuring the number of parameters in a network is one way of measuring the "size" of a network.

What is the total number of parameters in `small_net` and in `large_net`? (Hint: how many numbers are in each tensor?)

In [14]:
```
for param in small_net.parameters():
    print(param.shape)
```

```
torch.Size([5, 3, 3, 3])
torch.Size([5])
torch.Size([1, 245])
torch.Size([1])
```

1. torch.Size([5, 3, 3, 3]) are the Weights (Filters) for the Convolutional Layer. The Math: [Output Channels, Input Channels, Kernel Height, Kernel Width].

- 5 is the number of filters defined in nn.Conv2d(3, 5, ...).
- 3 is the number of input channels (Red, Green, Blue)
- 3, 3 defines a kernel size of 3, so each filter is 3×3.
- Total parameter count for torch.Size([5, 3, 3, 3]): $5 \times 3 \times 3 \times 3 = \mathbf{135}$ parameters

2. torch.Size([5])

- Parameter Count: $\mathbf{5}$ parameters.

3. torch.Size([1, 245])

- Parameter Count: $1 \times 245 = \mathbf{245}$ parameters.

4. torch.Size([1])

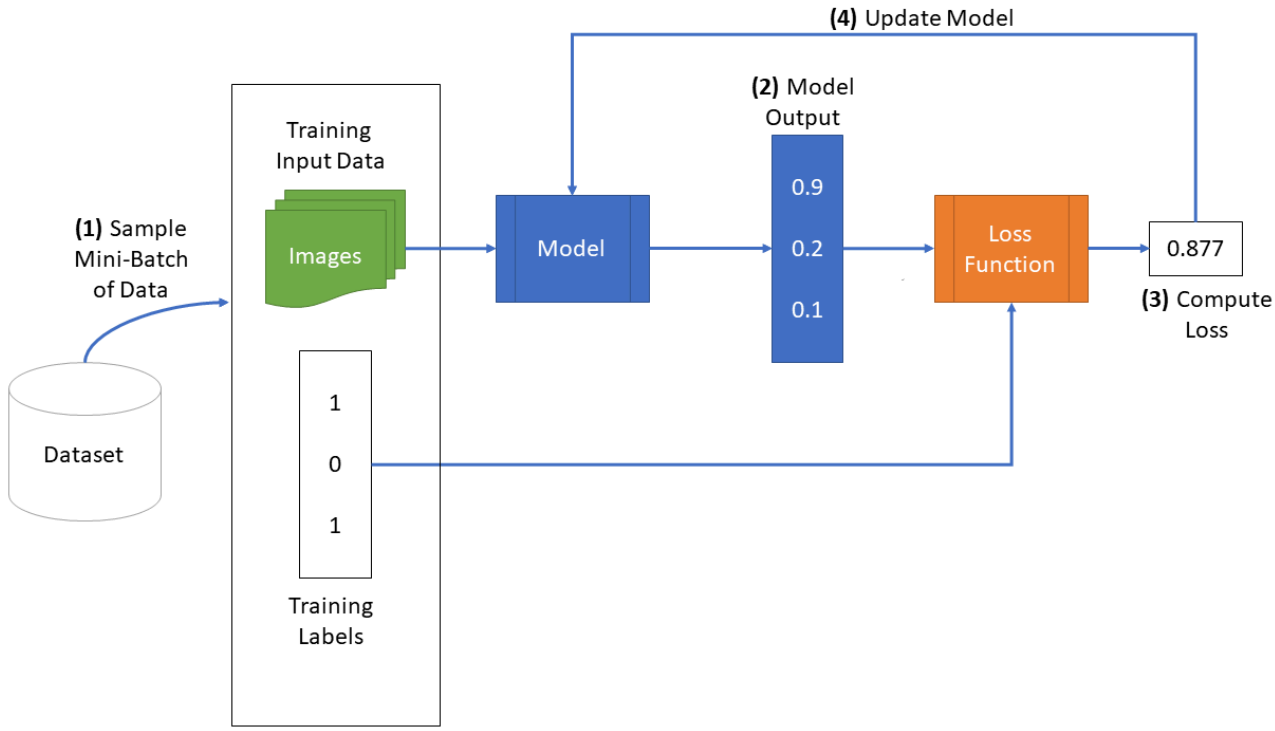
- Parameter Count: $\mathbf{1}$ parameter.

Total parameter count:

$$135 \text{ (Conv Weights)} + 5 \text{ (Conv Bias)} + 245 \text{ (FC Weights)} + 1 \text{ (FC Bias)} = \mathbf{386} \text{ parameters}$$

The function train_net

The function `train_net` below takes an untrained neural network (like `small_net` and `large_net`) and several other parameters. You should be able to understand how this function works. The figure below shows the high level training loop for a machine learning model:



In [16]:
```
def train_net(net, batch_size=64, learning_rate=0.01, num_epochs=30):
    ######################################################################
    # Train a classifier on cats vs dogs
    target_classes = ["cat", "dog"]
    ######################################################################
    # Fixed PyTorch random seed for reproducible result
    torch.manual_seed(1000)
    ######################################################################
    # Obtain the PyTorch data loader objects to load batches of the datasets
    train_loader, val_loader, test_loader, classes = get_data_loader(
            target_classes, batch_size)
    ######################################################################
    # Define the Loss function and optimizer
    # The loss function will be Binary Cross Entropy (BCE). In this case we
    # will use the BCEWithLogitsLoss which takes unnormalized output from
    # the neural network and scalar label.
    # Optimizer will be SGD with Momentum.
```

```
    criterion = nn.BCEWithLogitsLoss()
    optimizer = optim.SGD(net.parameters(), lr=learning_rate, momentum=0.9)
    ########################################################################
    # Set up some numpy arrays to store the training/test loss/erruracy
    train_err = np.zeros(num_epochs)
    train_loss = np.zeros(num_epochs)
    val_err = np.zeros(num_epochs)
    val_loss = np.zeros(num_epochs)
    ########################################################################
    # Train the network
    # Loop over the data iterator and sample a new batch of training data
    # Get the output from the network, and optimize our loss function.
    start_time = time.time()
    for epoch in range(num_epochs):  # loop over the dataset multiple times
        total_train_loss = 0.0
        total_train_err = 0.0
        total_epoch = 0
        for i, data in enumerate(train_loader, 0):
            # Get the inputs
            inputs, labels = data
            labels = normalize_label(labels) # Convert labels to 0/1
            # Zero the parameter gradients
            optimizer.zero_grad()
            # Forward pass, backward pass, and optimize
            outputs = net(inputs)
            loss = criterion(outputs, labels.float())
            loss.backward()
            optimizer.step()
            # Calculate the statistics
            corr = (outputs > 0.0).squeeze().long() != labels
            total_train_err += int(corr.sum())
            total_train_loss += loss.item()
            total_epoch += len(labels)
        train_err[epoch] = float(total_train_err) / total_epoch
        train_loss[epoch] = float(total_train_loss) / (i+1)
        val_err[epoch], val_loss[epoch] = evaluate(net, val_loader, criterion)
        print(("Epoch {}: Train err: {}, Train loss: {} |"+
               "Validation err: {}, Validation loss: {}").format(
                   epoch + 1,
                   train_err[epoch],
                   train_loss[epoch],
                   val_err[epoch],
                   val_loss[epoch]))
        # Save the current model (checkpoint) to a file
        model_path = get_model_name(net.name, batch_size, learning_rate, epoch)
        torch.save(net.state_dict(), model_path)
    print('Finished Training')
    end_time = time.time()
    elapsed_time = end_time - start_time
    print("Total time elapsed: {:.2f} seconds".format(elapsed_time))
    # Write the train/test loss/err into CSV file for plotting later
    epochs = np.arange(1, num_epochs + 1)
    np.savetxt("{}_train_err.csv".format(model_path), train_err)
    np.savetxt("{}_train_loss.csv".format(model_path), train_loss)
    np.savetxt("{}_val_err.csv".format(model_path), val_err)
    np.savetxt("{}_val_loss.csv".format(model_path), val_loss)
```

Part (b) -- 1pt

The parameters to the function `train_net` are hyperparameters of our neural network. We made these hyperparameters easy to modify so that we can tune them later on.

What are the default values of the parameters `batch_size`, `learning_rate`, and `num_epochs`?

def train_net(net, batch_size=64, learning_rate=0.01, num_epochs=30) is the definition of the function train_net. From here, it is obvious that the default values of the parameters batch_size is 64, learning_rate is 0.01 and num_epochs is 30.

Part (c) -- 3 pt

What files are written to disk when we call `train_net` with `small_net`, and train for 5 epochs? Provide a list of all the files written to disk, and what information the files contain.

The files written to the disk are:

- model_small_bs64_lr0.01_epoch0
- model_small_bs64_lr0.01_epoch1

- model_small_bs64_lr0.01_epoch2
- model_small_bs64_lr0.01_epoch3
- model_small_bs64_lr0.01_epoch4
- model_small_bs64_lr0.01_epoch4_train_err.csv
- model_small_bs64_lr0.01_epoch4_train_loss.csv
- model_small_bs64_lr0.01_epoch4_val_err.csv
- model_small_bs64_lr0.01_epoch4_val_loss.csv

The first 5 files that end with _epoch0,1,2,3,4 contain weights and biases of the neural network after that particular epoch. The last 4 CSV files contain training error, training loss, validation error and validation loss for each epoch respectively.

In [18]:
```
train_net(small_net,64, 0.01, 5)
```

```
Epoch 1: Train err: 0.41275, Train loss: 0.6701927328109741 |Validation err: 0.3825, Validation loss: 0.6588166169822216
Epoch 2: Train err: 0.369375, Train loss: 0.6473273797035217 |Validation err: 0.381, Validation loss: 0.6564719378948212
Epoch 3: Train err: 0.356625, Train loss: 0.635835352897644 |Validation err: 0.3435, Validation loss: 0.6275799535214901
Epoch 4: Train err: 0.3405, Train loss: 0.6193386876583099 |Validation err: 0.3555, Validation loss: 0.6283200923353434
Epoch 5: Train err: 0.331625, Train loss: 0.6094685463905335 |Validation err: 0.3305, Validation loss: 0.618465967476368
Finished Training
Total time elapsed: 23.85 seconds
```

Part (d) -- 2pt

Train both `small_net` and `large_net` using the function `train_net` and its default parameters. The function will write many files to disk, including a model checkpoint (saved values of model weights) at the end of each epoch.

If you are using Google Colab, you will need to mount Google Drive so that the files generated by `train_net` gets saved. We will be using these files in part (d). (See the Google Colab tutorial for more information about this.)

Report the total time elapsed when training each network. Which network took longer to train? Why?

In [19]:
```
# Since the function writes files to disk, you will need to mount
# your Google Drive. If you are working on the lab locally, you
# can comment out this code.

from google.colab import drive
drive.mount('/content/gdrive')
```

```
Mounted at /content/gdrive
```

In [20]:
```
train_net(small_net, 64, 0.01, 30)
train_net(large_net, 64, 0.01, 30)
```

```
Epoch 1: Train err: 0.322375, Train loss: 0.5987249422073364 |Validation err: 0.332, Validation loss: 0.6115447711199522
Epoch 2: Train err: 0.3125, Train loss: 0.5905107886791229 |Validation err: 0.3475, Validation loss: 0.6315749082714319
Epoch 3: Train err: 0.310875, Train loss: 0.5871077132225037 |Validation err: 0.322, Validation loss: 0.6002985490486026
Epoch 4: Train err: 0.308125, Train loss: 0.580567872285843 |Validation err: 0.321, Validation loss: 0.6055414397269487
Epoch 5: Train err: 0.3015, Train loss: 0.577467158794403 |Validation err: 0.3175, Validation loss: 0.6038832254707813
Epoch 6: Train err: 0.300625, Train loss: 0.5728600015640258 |Validation err: 0.3155, Validation loss: 0.6013897452503443
Epoch 7: Train err: 0.3005, Train loss: 0.5712952508926391 |Validation err: 0.321, Validation loss: 0.6012745844200253
Epoch 8: Train err: 0.294125, Train loss: 0.5681738257408142 |Validation err: 0.3115, Validation loss: 0.5943970000371337
Epoch 9: Train err: 0.294875, Train loss: 0.5687759244441986 |Validation err: 0.3145, Validation loss: 0.5954758878797293
Epoch 10: Train err: 0.290875, Train loss: 0.5628432700634003 |Validation err: 0.3135, Validation loss: 0.589256682433188
Epoch 11: Train err: 0.29, Train loss: 0.5630735092163086 |Validation err: 0.3185, Validation loss: 0.5910096485167742
Epoch 12: Train err: 0.289, Train loss: 0.5590385956764221 |Validation err: 0.315, Validation loss: 0.6028231494128704
Epoch 13: Train err: 0.292375, Train loss: 0.5594695386886597 |Validation err: 0.314, Validation loss: 0.5961981453001499
Epoch 14: Train err: 0.282375, Train loss: 0.555193089723587 |Validation err: 0.316, Validation loss: 0.603922426700592
Epoch 15: Train err: 0.283875, Train loss: 0.5559508161544799 |Validation err: 0.3165, Validation loss: 0.5895696179941297
Epoch 16: Train err: 0.287, Train loss: 0.5581419675350189 |Validation err: 0.3225, Validation loss: 0.6051628766581416
Epoch 17: Train err: 0.28225, Train loss: 0.5574386150836944 |Validation err: 0.311, Validation loss: 0.587420872412622
Epoch 18: Train err: 0.283, Train loss: 0.5509312105178833 |Validation err: 0.316, Validation loss: 0.5893959570676088
Epoch 19: Train err: 0.278375, Train loss: 0.5487329261302948 |Validation err: 0.327, Validation loss: 0.613205024972558
Epoch 20: Train err: 0.282375, Train loss: 0.5492946937084198 |Validation err: 0.315, Validation loss: 0.5956445634365082
Epoch 21: Train err: 0.280125, Train loss: 0.5532464623451233 |Validation err: 0.312, Validation loss: 0.5837026573717594
Epoch 22: Train err: 0.280875, Train loss: 0.5493385779857636 |Validation err: 0.3195, Validation loss: 0.5980055257678032
Epoch 23: Train err: 0.278375, Train loss: 0.5480739369392394 |Validation err: 0.309, Validation loss: 0.5828454960137606
Epoch 24: Train err: 0.278, Train loss: 0.5456148676872253 |Validation err: 0.314, Validation loss: 0.5879346048459411
Epoch 25: Train err: 0.27925, Train loss: 0.5442359421253204 |Validation err: 0.3025, Validation loss: 0.5840442972257733
Epoch 26: Train err: 0.270125, Train loss: 0.5442696959972382 |Validation err: 0.3165, Validation loss: 0.5975606804713607
Epoch 27: Train err: 0.272875, Train loss: 0.5427000482082367 |Validation err: 0.304, Validation loss: 0.5965242544189095
Epoch 28: Train err: 0.279, Train loss: 0.5480172505378723 |Validation err: 0.3055, Validation loss: 0.585088868625462
Epoch 29: Train err: 0.275625, Train loss: 0.5456124687194824 |Validation err: 0.3055, Validation loss: 0.588337229564786
Epoch 30: Train err: 0.275, Train loss: 0.54419588971138 |Validation err: 0.313, Validation loss: 0.5940384715795517
Finished Training
Total time elapsed: 145.60 seconds
Epoch 1: Train err: 0.456, Train loss: 0.6910030040740966 |Validation err: 0.426, Validation loss: 0.6817821320146322
Epoch 2: Train err: 0.418375, Train loss: 0.6771835770606994 |Validation err: 0.3965, Validation loss: 0.668139586225152
Epoch 3: Train err: 0.395375, Train loss: 0.6608083457946777 |Validation err: 0.3865, Validation loss: 0.6469494514167309
Epoch 4: Train err: 0.3705, Train loss: 0.6437877731323243 |Validation err: 0.3555, Validation loss: 0.636115163564682
Epoch 5: Train err: 0.352, Train loss: 0.6281496534347534 |Validation err: 0.3545, Validation loss: 0.62580331787467
Epoch 6: Train err: 0.335625, Train loss: 0.6131797881126404 |Validation err: 0.3405, Validation loss: 0.6220925096422434
Epoch 7: Train err: 0.323625, Train loss: 0.5984834976196289 |Validation err: 0.3255, Validation loss: 0.6054975017905235
Epoch 8: Train err: 0.31075, Train loss: 0.5831445388793945 |Validation err: 0.3145, Validation loss: 0.5949329454451799
Epoch 9: Train err: 0.30425, Train loss: 0.5727222979068756 |Validation err: 0.324, Validation loss: 0.5944483391940594
Epoch 10: Train err: 0.292875, Train loss: 0.5601065118312836 |Validation err: 0.3095, Validation loss: 0.5899208653718233
Epoch 11: Train err: 0.280125, Train loss: 0.545462218284607 |Validation err: 0.3165, Validation loss: 0.595277582295239
Epoch 12: Train err: 0.269875, Train loss: 0.533078152179718 |Validation err: 0.307, Validation loss: 0.5845487862825394
Epoch 13: Train err: 0.26375, Train loss: 0.524608728170395 |Validation err: 0.2995, Validation loss: 0.5927610918879509
Epoch 14: Train err: 0.258, Train loss: 0.5126013479232788 |Validation err: 0.3055, Validation loss: 0.5930404923856258
Epoch 15: Train err: 0.248875, Train loss: 0.5039637627601623 |Validation err: 0.307, Validation loss: 0.580553867854178
Epoch 16: Train err: 0.251125, Train loss: 0.5010448279380798 |Validation err: 0.3165, Validation loss: 0.5901796072721481
Epoch 17: Train err: 0.23975, Train loss: 0.48546155309677125 |Validation err: 0.299, Validation loss: 0.5988860120996833
Epoch 18: Train err: 0.230375, Train loss: 0.4712537355422974 |Validation err: 0.2935, Validation loss: 0.6033670613542199
Epoch 19: Train err: 0.224375, Train loss: 0.4629074125289917 |Validation err: 0.294, Validation loss: 0.596072381362319
Epoch 20: Train err: 0.21975, Train loss: 0.4538803730010986 |Validation err: 0.334, Validation loss: 0.7143326625227928
Epoch 21: Train err: 0.218625, Train loss: 0.45050936126708985 |Validation err: 0.2915, Validation loss: 0.5923697333782911
Epoch 22: Train err: 0.20325, Train loss: 0.4249889326095581 |Validation err: 0.3115, Validation loss: 0.6403741370886564
Epoch 23: Train err: 0.198125, Train loss: 0.42262443804740907 |Validation err: 0.289, Validation loss: 0.6087765153497458
Epoch 24: Train err: 0.1875, Train loss: 0.40796343398094176 |Validation err: 0.3005, Validation loss: 0.6622509406879544
Epoch 25: Train err: 0.179375, Train loss: 0.38845495009422304 |Validation err: 0.3025, Validation loss: 0.607969131320715
Epoch 26: Train err: 0.173875, Train loss: 0.3764057948589325 |Validation err: 0.304, Validation loss: 0.6769467676058412
Epoch 27: Train err: 0.167875, Train loss: 0.3631699361801147 |Validation err: 0.2985, Validation loss: 0.661359466612339
Epoch 28: Train err: 0.162375, Train loss: 0.35380494737625123 |Validation err: 0.2985, Validation loss: 0.7437802590429783
Epoch 29: Train err: 0.149875, Train loss: 0.3368068907260895 |Validation err: 0.3005, Validation loss: 0.7584704756736755
Epoch 30: Train err: 0.14925, Train loss: 0.32340435218811037 |Validation err: 0.2995, Validation loss: 0.7188131092116237
Finished Training
Total time elapsed: 158.55 seconds
```

Part (e) - 2pt

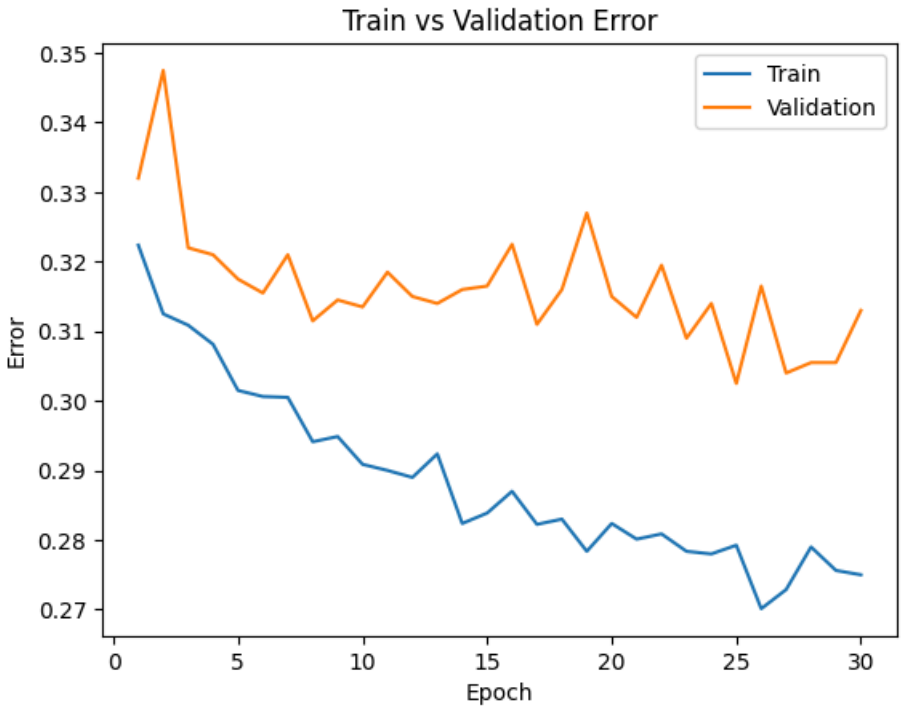
Use the function `plot_training_curve` to display the trajectory of the training/validation error and the training/validation loss. You will need to use the function `get_model_name` to generate the argument to the `plot_training_curve` function.

Do this for both the small network and the large network. Include both plots in your writeup.

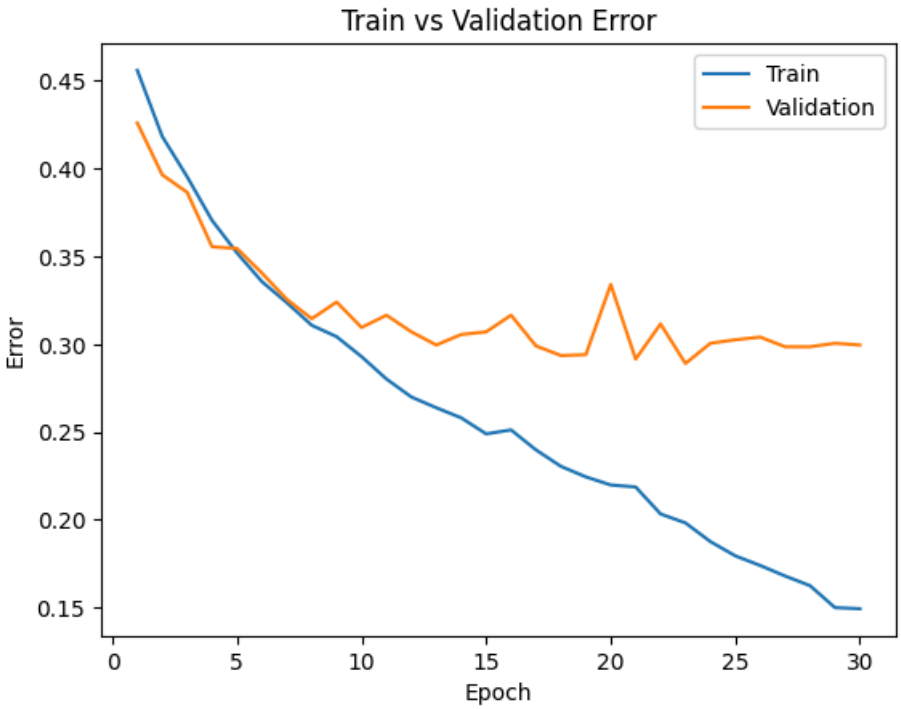
In [25]:
```
#model_path = get_model_name("small", batch_size=??, learning_rate=??, epoch=29)
print("Graphs for small_net")
model_path_small = get_model_name("small", batch_size=64, learning_rate=0.01, epoch=29)
plot_training_curve(model_path_small)

print("Graphs for large_net")
```

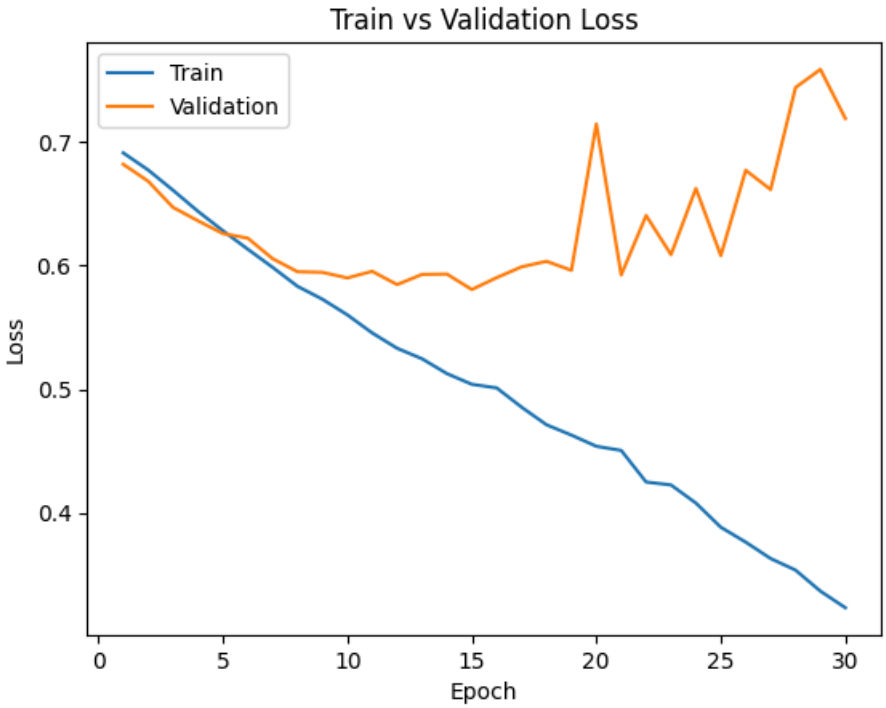
```
model_path_large = get_model_name("large", batch_size=64, learning_rate=0.01, epoch=29)
plot_training_curve(model_path_large)
```

Graphs for small_net



Graphs for large_net





Part (f) - 5pt

Describe what you notice about the training curve. How do the curves differ for `small_net` and `large_net`? Identify any occurences of underfitting and overfitting.

The large_net demonstrates clear overfitting. While it achieves a much lower training error than small_net (indicating better learning capacity), the validation loss begins to diverge and increase rapidly after approximately Epoch 15. This indicates that the additional parameters in large_net allowed it to memorize the noise in the training set, harming its ability to generalize to new data. In contrast, small_net had higher bias (higher training error) but lower variance (less gap between train/val).

Part 3. Optimization Parameters [12 pt]

For this section, we will work with `large_net` only.

Part (a) - 3pt

Train `large_net` with all default parameters, except set `learning_rate=0.001`. Does the model take longer/shorter to train? Plot the training curve. Describe the effect of *lowering* the learning rate.

In [26]:
```
# Note: When we re-construct the model, we start the training
# with *random weights*. If we omit this code, the values of
```

```
# the weights will still be the previously trained values.
large_net = LargeNet()
train_net(large_net, 64, 0.001, 30)
```

```
Epoch 1: Train err: 0.47625, Train loss: 0.6928360004425049 |Validation err: 0.467, Validation loss: 0.6924686580896378
Epoch 2: Train err: 0.448625, Train loss: 0.6922589716911316 |Validation err: 0.4305, Validation loss: 0.6916493363678455
Epoch 3: Train err: 0.43575, Train loss: 0.6916067404747009 |Validation err: 0.4285, Validation loss: 0.6908544525504112
Epoch 4: Train err: 0.430125, Train loss: 0.6908613877296448 |Validation err: 0.424, Validation loss: 0.68965969607234
Epoch 5: Train err: 0.434125, Train loss: 0.6899198365211486 |Validation err: 0.4195, Validation loss: 0.688694279640913
Epoch 6: Train err: 0.435875, Train loss: 0.6887419748306275 |Validation err: 0.4195, Validation loss: 0.6867837496101856
Epoch 7: Train err: 0.436625, Train loss: 0.6873781814575195 |Validation err: 0.4185, Validation loss: 0.6851996649056673
Epoch 8: Train err: 0.43725, Train loss: 0.6859267811775207 |Validation err: 0.4115, Validation loss: 0.6831991244107485
Epoch 9: Train err: 0.424375, Train loss: 0.6844044809341431 |Validation err: 0.411, Validation loss: 0.6808866038918495
Epoch 10: Train err: 0.42425, Train loss: 0.6828490204811096 |Validation err: 0.408, Validation loss: 0.678350143134594
Epoch 11: Train err: 0.425375, Train loss: 0.6812362918853759 |Validation err: 0.4125, Validation loss: 0.6780207101255655
Epoch 12: Train err: 0.420125, Train loss: 0.6796324462890625 |Validation err: 0.4125, Validation loss: 0.6753147002309561
Epoch 13: Train err: 0.414875, Train loss: 0.6777921686172486 |Validation err: 0.415, Validation loss: 0.6757054049521685
Epoch 14: Train err: 0.41225, Train loss: 0.6761114087104797 |Validation err: 0.412, Validation loss: 0.67397416010499
Epoch 15: Train err: 0.409, Train loss: 0.6744724254608154 |Validation err: 0.415, Validation loss: 0.6706814523786306
Epoch 16: Train err: 0.406375, Train loss: 0.6727445869445801 |Validation err: 0.4105, Validation loss: 0.6707715895026922
Epoch 17: Train err: 0.401375, Train loss: 0.67130890417099 |Validation err: 0.404, Validation loss: 0.6671548075973988
Epoch 18: Train err: 0.399375, Train loss: 0.6696756863594056 |Validation err: 0.4055, Validation loss: 0.6646822784096003
Epoch 19: Train err: 0.40075, Train loss: 0.6679100017547608 |Validation err: 0.396, Validation loss: 0.6655073687434196
Epoch 20: Train err: 0.39225, Train loss: 0.665790048122406 |Validation err: 0.4045, Validation loss: 0.6626051049679518
Epoch 21: Train err: 0.389625, Train loss: 0.6646309685707092 |Validation err: 0.3945, Validation loss: 0.6606833804398775
Epoch 22: Train err: 0.38875, Train loss: 0.6623744034767151 |Validation err: 0.393, Validation loss: 0.6617004871368408
Epoch 23: Train err: 0.384125, Train loss: 0.6601551241874695 |Validation err: 0.3975, Validation loss: 0.6574001368135214
Epoch 24: Train err: 0.382375, Train loss: 0.6584072341918945 |Validation err: 0.3865, Validation loss: 0.6561403777450323
Epoch 25: Train err: 0.378875, Train loss: 0.6555052490234375 |Validation err: 0.3885, Validation loss: 0.6552855931222439
Epoch 26: Train err: 0.37675, Train loss: 0.6531328277587891 |Validation err: 0.387, Validation loss: 0.6531829237937927
Epoch 27: Train err: 0.375125, Train loss: 0.6503917617797852 |Validation err: 0.387, Validation loss: 0.6519917994737625
Epoch 28: Train err: 0.371375, Train loss: 0.647677414894104 |Validation err: 0.3875, Validation loss: 0.6483855955302715
Epoch 29: Train err: 0.368, Train loss: 0.6451585369110108 |Validation err: 0.382, Validation loss: 0.6459537353366613
Epoch 30: Train err: 0.362875, Train loss: 0.6423822708129883 |Validation err: 0.3785, Validation loss: 0.6439289432018995
Finished Training
Total time elapsed: 152.92 seconds
```

Part (b) - 3pt

Train `large_net` with all default parameters, except set `learning_rate=0.1`. Does the model take longer/shorter to train? Plot the training curve. Describe the effect of *increasing* the learning rate.

In [30]:
```
(large_net) = LargeNet()
train_net(large_net, 64, 0.1, 30)
```

```
Epoch 1: Train err: 0.4295, Train loss: 0.6743808445930481 |Validation err: 0.3565, Validation loss: 0.6350402887910604
Epoch 2: Train err: 0.36925, Train loss: 0.6415545463562011 |Validation err: 0.3515, Validation loss: 0.6243696268647909
Epoch 3: Train err: 0.366, Train loss: 0.6304733338356018 |Validation err: 0.3475, Validation loss: 0.6173483617603779
Epoch 4: Train err: 0.35775, Train loss: 0.6251179370880127 |Validation err: 0.3345, Validation loss: 0.620793430134654
Epoch 5: Train err: 0.33675, Train loss: 0.6045795600414277 |Validation err: 0.329, Validation loss: 0.5937281344085932
Epoch 6: Train err: 0.31775, Train loss: 0.5828781847953797 |Validation err: 0.3275, Validation loss: 0.6041664052754641
Epoch 7: Train err: 0.3195, Train loss: 0.5794640259742737 |Validation err: 0.334, Validation loss: 0.6052154619246721
Epoch 8: Train err: 0.311875, Train loss: 0.5757701637744903 |Validation err: 0.318, Validation loss: 0.5993684772402048
Epoch 9: Train err: 0.29325, Train loss: 0.5609976608753204 |Validation err: 0.3245, Validation loss: 0.5896207233890891
Epoch 10: Train err: 0.28325, Train loss: 0.5477550032138825 |Validation err: 0.3275, Validation loss: 0.5987504478543997
Epoch 11: Train err: 0.272375, Train loss: 0.5326508040428162 |Validation err: 0.311, Validation loss: 0.6202037688344717
Epoch 12: Train err: 0.25825, Train loss: 0.5120045669078827 |Validation err: 0.313, Validation loss: 0.6050943713635206
Epoch 13: Train err: 0.262375, Train loss: 0.5099549624919891 |Validation err: 0.328, Validation loss: 0.6595966126769781
Epoch 14: Train err: 0.2605, Train loss: 0.508032098531723 |Validation err: 0.3245, Validation loss: 0.7022156557068229
Epoch 15: Train err: 0.246125, Train loss: 0.4948580477237701 |Validation err: 0.3245, Validation loss: 0.6181976906955242
Epoch 16: Train err: 0.25425, Train loss: 0.5112269511222839 |Validation err: 0.3365, Validation loss: 0.6773550705984235
Epoch 17: Train err: 0.256, Train loss: 0.5096174550056457 |Validation err: 0.3215, Validation loss: 0.6484736818820238
Epoch 18: Train err: 0.241875, Train loss: 0.48050818586349486 |Validation err: 0.3245, Validation loss: 0.644468174315989
Epoch 19: Train err: 0.24475, Train loss: 0.497201379776001 |Validation err: 0.322, Validation loss: 0.6747357528656721
Epoch 20: Train err: 0.2445, Train loss: 0.4948779764175415 |Validation err: 0.3285, Validation loss: 0.7009244337677956
Epoch 21: Train err: 0.245375, Train loss: 0.49160154938697814 |Validation err: 0.3445, Validation loss: 0.698606455698609
4
Epoch 22: Train err: 0.244, Train loss: 0.48209749364852905 |Validation err: 0.3275, Validation loss: 0.7424729950726032
Epoch 23: Train err: 0.235625, Train loss: 0.4757409155368804 7 |Validation err: 0.337, Validation loss: 0.7379261311143637
Epoch 24: Train err: 0.2195, Train loss: 0.45225173568725585 |Validation err: 0.3055, Validation loss: 0.8094820464029908
Epoch 25: Train err: 0.216375, Train loss: 0.44918966364860535 |Validation err: 0.3615, Validation loss: 0.775574964471161
4
Epoch 26: Train err: 0.229875, Train loss: 0.46733192253112793 |Validation err: 0.3885, Validation loss: 0.746215553954243
7
Epoch 27: Train err: 0.244875, Train loss: 0.4980352530479431 |Validation err: 0.3535, Validation loss: 0.7864172533154488
Epoch 28: Train err: 0.22475, Train loss: 0.4615201361179352 |Validation err: 0.335, Validation loss: 0.8736669421195984
Epoch 29: Train err: 0.233625, Train loss: 0.4676371040344238 |Validation err: 0.348, Validation loss: 0.7977995797991753
Epoch 30: Train err: 0.235, Train loss: 0.4715475182533264 |Validation err: 0.361, Validation loss: 0.8199478834867477
Finished Training
Total time elapsed: 157.23 seconds
```

The training time was approximately similar (~152s vs ~157s). The slight difference is likely due to system load variance or network latency, as the computational complexity of the backpropagation update remains constant regardless of the learning rate scalar. The time taken for each epoch should be the same regardless of learning rate and since the number of epoches and other values are set to default, there is no effect on the training time.

Part (c) - 3pt

Train `large_net` with all default parameters, including with `learning_rate=0.01`. Now, set `batch_size=512`. Does the model take longer/shorter to train? Plot the training curve. Describe the effect of *increasing* the batch size.

In [28]:
```
(large_net) = LargeNet()
train_net(large_net, 512, 0.01, 30)
```

```
Epoch 1: Train err: 0.48175, Train loss: 0.6929379552602768 |Validation err: 0.478, Validation loss: 0.6926824003458023
Epoch 2: Train err: 0.457625, Train loss: 0.692410409450531 |Validation err: 0.434, Validation loss: 0.6917425245046616
Epoch 3: Train err: 0.437, Train loss: 0.6916500627994537 |Validation err: 0.4265, Validation loss: 0.6909130215644836
Epoch 4: Train err: 0.433625, Train loss: 0.6908449903130531 |Validation err: 0.424, Validation loss: 0.6897870302200317
Epoch 5: Train err: 0.434, Train loss: 0.6896935515105724 |Validation err: 0.424, Validation loss: 0.6881355047225952
Epoch 6: Train err: 0.438, Train loss: 0.6883532106876373 |Validation err: 0.4285, Validation loss: 0.686011865735054
Epoch 7: Train err: 0.439375, Train loss: 0.6866871826350689 |Validation err: 0.426, Validation loss: 0.6836968660354614
Epoch 8: Train err: 0.43525, Train loss: 0.6849770620465279 |Validation err: 0.411, Validation loss: 0.6814672648906708
Epoch 9: Train err: 0.42375, Train loss: 0.6832008883357048 |Validation err: 0.414, Validation loss: 0.6795914173126221
Epoch 10: Train err: 0.421, Train loss: 0.6811088025569916 |Validation err: 0.416, Validation loss: 0.6771541684865952
Epoch 11: Train err: 0.42075, Train loss: 0.6794030703604221 |Validation err: 0.4095, Validation loss: 0.6748128235340118
Epoch 12: Train err: 0.41475, Train loss: 0.6768062897026539 |Validation err: 0.412, Validation loss: 0.6737067103385925
Epoch 13: Train err: 0.41025, Train loss: 0.6749698966741562 |Validation err: 0.412, Validation loss: 0.6706121861934662
Epoch 14: Train err: 0.407125, Train loss: 0.6730909235775471 |Validation err: 0.4125, Validation loss: 0.6692100465297699
Epoch 15: Train err: 0.4005, Train loss: 0.6706824786961079 |Validation err: 0.41, Validation loss: 0.6672518998384476
Epoch 16: Train err: 0.397625, Train loss: 0.669177521020174 |Validation err: 0.405, Validation loss: 0.6649047136306763
Epoch 17: Train err: 0.393875, Train loss: 0.6675704158842564 |Validation err: 0.401, Validation loss: 0.6630221307277679
Epoch 18: Train err: 0.393, Train loss: 0.6648007929325104 |Validation err: 0.3945, Validation loss: 0.6623915135860443
Epoch 19: Train err: 0.386125, Train loss: 0.6627416461706161 |Validation err: 0.3875, Validation loss: 0.6597241759300232
Epoch 20: Train err: 0.38175, Train loss: 0.659615233540535 |Validation err: 0.401, Validation loss: 0.6564163863658905
Epoch 21: Train err: 0.385875, Train loss: 0.6584861390292645 |Validation err: 0.388, Validation loss: 0.6586255580186844
Epoch 22: Train err: 0.378625, Train loss: 0.6551117822527885 |Validation err: 0.3855, Validation loss: 0.6528623402118683
Epoch 23: Train err: 0.371125, Train loss: 0.6508784741163254 |Validation err: 0.3835, Validation loss: 0.6498105823993683
Epoch 24: Train err: 0.37675, Train loss: 0.6488191038370132 |Validation err: 0.384, Validation loss: 0.647487610578537
Epoch 25: Train err: 0.369, Train loss: 0.6445828042924404 |Validation err: 0.3825, Validation loss: 0.6473540216684341
Epoch 26: Train err: 0.373, Train loss: 0.6428647711873055 |Validation err: 0.375, Validation loss: 0.6425962299108505
Epoch 27: Train err: 0.35975, Train loss: 0.6372342072427273 |Validation err: 0.378, Validation loss: 0.6397327333688736
Epoch 28: Train err: 0.354125, Train loss: 0.6337889917194843 |Validation err: 0.37, Validation loss: 0.6403442025184631
Epoch 29: Train err: 0.353875, Train loss: 0.6311212778091431 |Validation err: 0.3665, Validation loss: 0.6335662603378296
Epoch 30: Train err: 0.35275, Train loss: 0.6283531747758389 |Validation err: 0.367, Validation loss: 0.6324412226676941
Finished Training
Total time elapsed: 139.36 seconds
```

While keeping other default parameters, increasing the batch size will result in the model taking shorter time to train. This can be seen by the training experiments above as the time to train with batch size= 64 is 152.92 seconds while the time to train with batch size= 512 is 139.36 seconds.

Part (d) - 3pt

Train `large_net` with all default parameters, including with `learning_rate=0.01`. Now, set `batch_size=16`. Does the model take longer/shorter to train? Plot the training curve. Describe the effect of *decreasing* the batch size.

In [29]:
```
(large_net) = LargeNet()
train_net(large_net, 16, 0.01, 30)
```

```
Epoch 1: Train err: 0.432625, Train loss: 0.6775506126880646 |Validation err: 0.378, Validation loss: 0.6512571973800659
Epoch 2: Train err: 0.366375, Train loss: 0.6387728816270828 |Validation err: 0.337, Validation loss: 0.612742235660553
Epoch 3: Train err: 0.339375, Train loss: 0.6119522891640663 |Validation err: 0.345, Validation loss: 0.6396356213092804
Epoch 4: Train err: 0.312125, Train loss: 0.5861616842746734 |Validation err: 0.3375, Validation loss: 0.6223122742176056
Epoch 5: Train err: 0.309125, Train loss: 0.5655454085469246 |Validation err: 0.303, Validation loss: 0.5682719912528992
Epoch 6: Train err: 0.283875, Train loss: 0.546434996843338 |Validation err: 0.3125, Validation loss: 0.581913836479187
Epoch 7: Train err: 0.275625, Train loss: 0.5315411986708641 |Validation err: 0.3185, Validation loss: 0.5755203785896301
Epoch 8: Train err: 0.253625, Train loss: 0.5110043309926987 |Validation err: 0.326, Validation loss: 0.6016001827716827
Epoch 9: Train err: 0.249375, Train loss: 0.4988122125864029 |Validation err: 0.301, Validation loss: 0.5766945521831512
Epoch 10: Train err: 0.234375, Train loss: 0.47769227081537247 |Validation err: 0.2935, Validation loss: 0.5778298568725586
Epoch 11: Train err: 0.2245, Train loss: 0.4637914804518223 |Validation err: 0.295, Validation loss: 0.6023752751350403
Epoch 12: Train err: 0.208625, Train loss: 0.4436814887523651 |Validation err: 0.3015, Validation loss: 0.5988463146686553
Epoch 13: Train err: 0.1995, Train loss: 0.42190410897135733 |Validation err: 0.2965, Validation loss: 0.6023776261806488
Epoch 14: Train err: 0.18225, Train loss: 0.3964607211649418 |Validation err: 0.285, Validation loss: 0.6563077869415284
Epoch 15: Train err: 0.176625, Train loss: 0.38504845155775547 |Validation err: 0.3015, Validation loss: 0.6845730379819871
Epoch 16: Train err: 0.160125, Train loss: 0.35515162971615793 |Validation err: 0.313, Validation loss: 0.7153961782455445
Epoch 17: Train err: 0.146875, Train loss: 0.33127270932495595 |Validation err: 0.3095, Validation loss: 0.8292664550542831
Epoch 18: Train err: 0.1435, Train loss: 0.3216609486192465 |Validation err: 0.3265, Validation loss: 0.8005202133655548
Epoch 19: Train err: 0.130375, Train loss: 0.30764051334559916 |Validation err: 0.3255, Validation loss: 0.8987283756732941
Epoch 20: Train err: 0.11675, Train loss: 0.28437252435833216 |Validation err: 0.309, Validation loss: 0.8813227322101593
Epoch 21: Train err: 0.121625, Train loss: 0.27901479678601027 |Validation err: 0.3175, Validation loss: 0.8441186798810959
Epoch 22: Train err: 0.112875, Train loss: 0.2728129132091999 |Validation err: 0.3125, Validation loss: 0.8382637614011764
Epoch 23: Train err: 0.1, Train loss: 0.24255631332099437 |Validation err: 0.3265, Validation loss: 1.0690370055437088
Epoch 24: Train err: 0.095375, Train loss: 0.2318725396282971 |Validation err: 0.322, Validation loss: 1.0889509381055833
Epoch 25: Train err: 0.095875, Train loss: 0.23257503168098628 |Validation err: 0.329, Validation loss: 1.0898759965896607
Epoch 26: Train err: 0.08275, Train loss: 0.20350570978596808 |Validation err: 0.321, Validation loss: 1.127944498538971
Epoch 27: Train err: 0.09075, Train loss: 0.2147584371343255 |Validation err: 0.3205, Validation loss: 1.1985667142868042
Epoch 28: Train err: 0.076125, Train loss: 0.18171320636058227 |Validation err: 0.322, Validation loss: 1.3782274899482727
Epoch 29: Train err: 0.091875, Train loss: 0.21968170262500644 |Validation err: 0.3245, Validation loss: 1.2601391798257828
Epoch 30: Train err: 0.07725, Train loss: 0.1865569370612502 |Validation err: 0.323, Validation loss: 1.3147952773571014
Finished Training
Total time elapsed: 228.34 seconds
```

While keeping other default parameters, decreasing the batch size will result in the model taking longer time to train. This can be seen by the training experiments above as the time to train with batch size= 64 is 152.92 seconds while the time to train with batch size= 16 is 228.34 seconds.

Part 4. Hyperparameter Search [6 pt]

Part (a) - 2pt

Based on the plots from above, choose another set of values for the hyperparameters (network, batch_size, learning_rate) that you think would help you improve the validation accuracy. Justify your choice.

Choices:

- Network: LargeNet
- Batch Size: 64
- Learning Rate: 0.005

Justification: I selected LargeNet with a batch size of 64 and a learning rate of 0.005. LargeNet was chosen because it demonstrated superior learning capacity, achieving a final training error of ~15% compared to SmallNet's ~28%. For the learning rate, the logs revealed that 0.001 resulted in underfitting (stuck at ~36% error), while 0.01 and 0.1 caused instability and overfitting where the validation loss exploded or oscillated. A rate of 0.005 serves as the optimal middle ground, offering enough speed to converge better than 0.001 while being small enough to stabilize the training updates and prevent the overfitting seen at 0.01.

Part (b) - 1pt

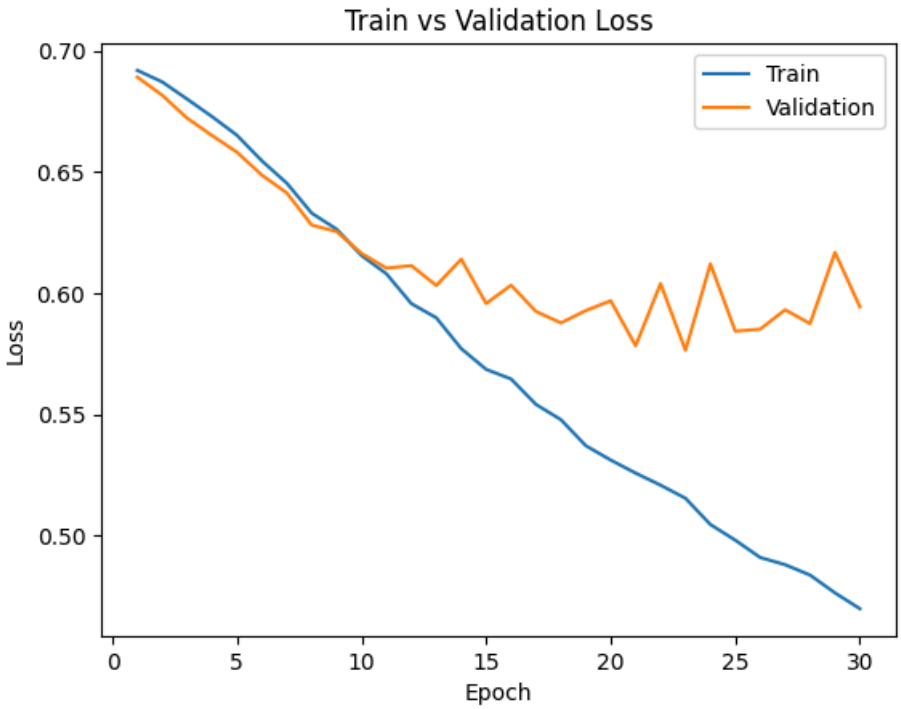
Train the model with the hyperparameters you chose in part(a), and include the training curve.

In [31]:
```
(large_net) = LargeNet()
train_net(large_net, 64, 0.005, 30)
```

```
Epoch 1: Train err: 0.4525, Train loss: 0.691849802494049 |Validation err: 0.429, Validation loss: 0.6890992522239685
Epoch 2: Train err: 0.441375, Train loss: 0.6870906562805176 |Validation err: 0.4175, Validation loss: 0.6815983336418867
Epoch 3: Train err: 0.419125, Train loss: 0.6800039582252503 |Validation err: 0.414, Validation loss: 0.6721466891467571
Epoch 4: Train err: 0.407125, Train loss: 0.6727867360115052 |Validation err: 0.399, Validation loss: 0.6649869475513697
Epoch 5: Train err: 0.393, Train loss: 0.6650896525382995 |Validation err: 0.39, Validation loss: 0.6581201199442148
Epoch 6: Train err: 0.375625, Train loss: 0.6545388355255127 |Validation err: 0.376, Validation loss: 0.6486515775322914
Epoch 7: Train err: 0.373125, Train loss: 0.6453463292121887 |Validation err: 0.377, Validation loss: 0.6413232330232859
Epoch 8: Train err: 0.357875, Train loss: 0.632988317489624 |Validation err: 0.3565, Validation loss: 0.628065874800086
Epoch 9: Train err: 0.351625, Train loss: 0.6263490128517151 |Validation err: 0.346, Validation loss: 0.6254121940582991
Epoch 10: Train err: 0.34375, Train loss: 0.6154989910125732 |Validation err: 0.3325, Validation loss: 0.6163706928491592
Epoch 11: Train err: 0.3305, Train loss: 0.607956873178482 |Validation err: 0.336, Validation loss: 0.6103206016123295
Epoch 12: Train err: 0.32, Train loss: 0.5956837391853332 |Validation err: 0.3435, Validation loss: 0.6113488059490919
Epoch 13: Train err: 0.31725, Train loss: 0.5898542129993438 |Validation err: 0.3265, Validation loss: 0.6031631994992495
Epoch 14: Train err: 0.307875, Train loss: 0.5771440901756286 |Validation err: 0.333, Validation loss: 0.6139950659126043
Epoch 15: Train err: 0.2965, Train loss: 0.5685857439041138 |Validation err: 0.315, Validation loss: 0.5957461772486567
Epoch 16: Train err: 0.2945, Train loss: 0.5646279911994934 |Validation err: 0.3175, Validation loss: 0.6032675551250577
Epoch 17: Train err: 0.29, Train loss: 0.5541331174373627 |Validation err: 0.309, Validation loss: 0.5924727525562048
Epoch 18: Train err: 0.283, Train loss: 0.5478503708839416 |Validation err: 0.314, Validation loss: 0.587733949534595
Epoch 19: Train err: 0.27325, Train loss: 0.5371154701709747 |Validation err: 0.316, Validation loss: 0.592813877388835
Epoch 20: Train err: 0.269125, Train loss: 0.5311978349685669 |Validation err: 0.303, Validation loss: 0.5968699203804135
Epoch 21: Train err: 0.27025, Train loss: 0.5258198037147522 |Validation err: 0.3015, Validation loss: 0.5783241260796785
Epoch 22: Train err: 0.266125, Train loss: 0.5208520097732544 |Validation err: 0.3145, Validation loss: 0.6039246218279004
Epoch 23: Train err: 0.258875, Train loss: 0.5154560666084289 |Validation err: 0.304, Validation loss: 0.576459183357656
Epoch 24: Train err: 0.25375, Train loss: 0.5046343743801117 |Validation err: 0.326, Validation loss: 0.6120715029537678
Epoch 25: Train err: 0.249, Train loss: 0.4981588492393494 |Validation err: 0.306, Validation loss: 0.5843336805701256
Epoch 26: Train err: 0.247, Train loss: 0.4910268070697784 |Validation err: 0.3055, Validation loss: 0.5851457724347711
Epoch 27: Train err: 0.24075, Train loss: 0.48807174849510193 |Validation err: 0.2975, Validation loss: 0.5931426426395774
Epoch 28: Train err: 0.2385, Train loss: 0.48378628182411193 |Validation err: 0.312, Validation loss: 0.5874433973804116
Epoch 29: Train err: 0.234125, Train loss: 0.4764461867809296 |Validation err: 0.3245, Validation loss: 0.6167989503592253
Epoch 30: Train err: 0.229375, Train loss: 0.46991838145256043 |Validation err: 0.308, Validation loss: 0.5943732624873519
Finished Training
Total time elapsed: 162.69 seconds
```

In [33]:

```
model_path = get_model_name("large", batch_size=64, learning_rate=0.005, epoch=29)
plot_training_curve(model_path)
```



Part (c) - 2pt

Based on your result from Part(a), suggest another set of hyperparameter values to try. Justify your choice.

Choices:

- Network: LargeNet
- Batch Size: 32
- Learning Rate: 0.001

Justification: In Part (a), we saw that a learning rate of 0.001 was too slow when paired with a batch size of 64, resulting in underfitting (high bias). Instead of increasing the learning rate—which previously caused instability at 0.01—I chose to decrease the batch size to 32. This effectively doubles the number of weight updates per epoch (since there are twice as many batches). This change should accelerate convergence and help the model escape the underfitting zone, while still maintaining the precise, stable updates of the lower learning rate.

Part (d) - 1pt

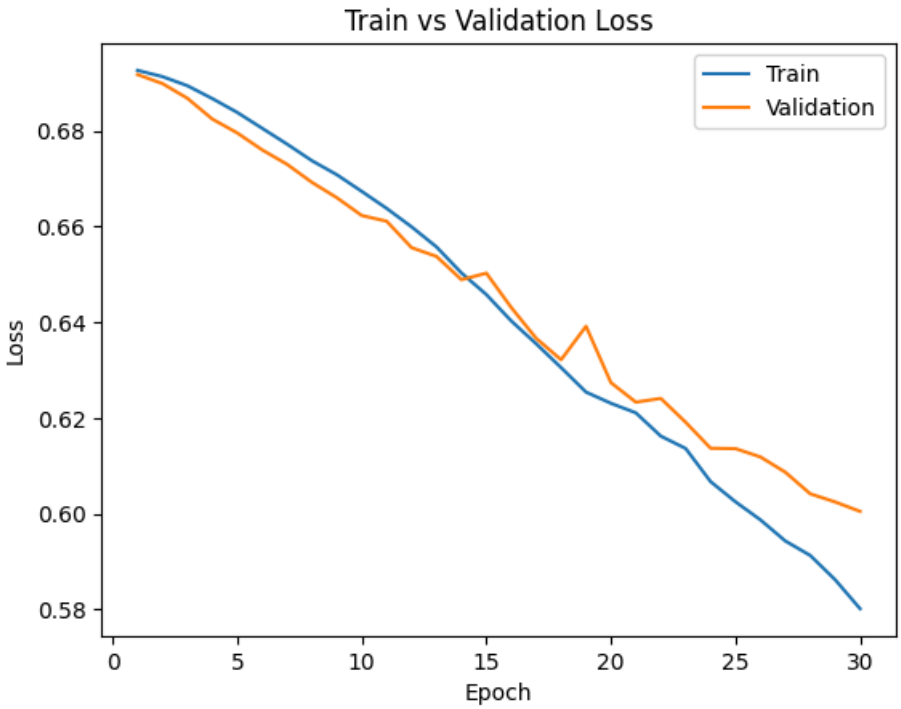
Train the model with the hyperparameters you chose in part(c), and include the training curve.

In [34]:
```
(large_net) = LargeNet()
train_net(large_net, 32, 0.001, 30)
```

```
Epoch 1: Train err: 0.466125, Train loss: 0.6925730061531067 |Validation err: 0.4335, Validation loss: 0.6916963685126531
Epoch 2: Train err: 0.447, Train loss: 0.6912713127136231 |Validation err: 0.4235, Validation loss: 0.6898154786654881
Epoch 3: Train err: 0.43475, Train loss: 0.6893712038993836 |Validation err: 0.418, Validation loss: 0.6867533345071096
Epoch 4: Train err: 0.433, Train loss: 0.6866875042915345 |Validation err: 0.42, Validation loss: 0.6824125420479548
Epoch 5: Train err: 0.42575, Train loss: 0.6837978281974793 |Validation err: 0.4125, Validation loss: 0.6795159143114847
Epoch 6: Train err: 0.418625, Train loss: 0.6804902694225311 |Validation err: 0.4115, Validation loss: 0.67598110058951
Epoch 7: Train err: 0.412125, Train loss: 0.6771817028522491 |Validation err: 0.414, Validation loss: 0.6729649939234295
Epoch 8: Train err: 0.40925, Train loss: 0.6737111871242523 |Validation err: 0.41, Validation loss: 0.6691891077965025
Epoch 9: Train err: 0.402625, Train loss: 0.6707859535217285 |Validation err: 0.4075, Validation loss: 0.6659819502679128
Epoch 10: Train err: 0.39725, Train loss: 0.6673015077114105 |Validation err: 0.4025, Validation loss: 0.6622666328672379
Epoch 11: Train err: 0.392, Train loss: 0.6637524721622468 |Validation err: 0.3925, Validation loss: 0.6610736809079609
Epoch 12: Train err: 0.385625, Train loss: 0.659901665687561 |Validation err: 0.392, Validation loss: 0.6555543522986155
Epoch 13: Train err: 0.3805, Train loss: 0.6557084238529205 |Validation err: 0.3845, Validation loss: 0.6537226996724568
Epoch 14: Train err: 0.372, Train loss: 0.650277382850647 |Validation err: 0.3835, Validation loss: 0.6488760246170892
Epoch 15: Train err: 0.37525, Train loss: 0.6457659413814545 |Validation err: 0.3835, Validation loss: 0.6502270509326269
Epoch 16: Train err: 0.365875, Train loss: 0.64025097489357 |Validation err: 0.382, Validation loss: 0.6430550709603324
Epoch 17: Train err: 0.359, Train loss: 0.6354857084751129 |Validation err: 0.3685, Validation loss: 0.6366187882801843
Epoch 18: Train err: 0.3575, Train loss: 0.6305058262348175 |Validation err: 0.3615, Validation loss: 0.6321532125510867
Epoch 19: Train err: 0.349, Train loss: 0.625401654958725 |Validation err: 0.371, Validation loss: 0.6391485645657494
Epoch 20: Train err: 0.347375, Train loss: 0.6230582429170608 |Validation err: 0.358, Validation loss: 0.6273676970648387
Epoch 21: Train err: 0.34425, Train loss: 0.6210695291757584 |Validation err: 0.3485, Validation loss: 0.6232976033574059
Epoch 22: Train err: 0.343, Train loss: 0.6162130485773086 |Validation err: 0.345, Validation loss: 0.6240808306232332
Epoch 23: Train err: 0.339, Train loss: 0.6136242228746415 |Validation err: 0.339, Validation loss: 0.6191000261950115
Epoch 24: Train err: 0.3375, Train loss: 0.6067070052623749 |Validation err: 0.337, Validation loss: 0.6136740592737047
Epoch 25: Train err: 0.335375, Train loss: 0.6024892760515212 |Validation err: 0.343, Validation loss: 0.6135907362377833
Epoch 26: Train err: 0.3255, Train loss: 0.5987290207147599 |Validation err: 0.3375, Validation loss: 0.6118580117112115
Epoch 27: Train err: 0.326125, Train loss: 0.5943105072975159 |Validation err: 0.321, Validation loss: 0.6086676082913838
Epoch 28: Train err: 0.321875, Train loss: 0.5912907313108444 |Validation err: 0.3195, Validation loss: 0.6041646528811682
Epoch 29: Train err: 0.315875, Train loss: 0.5862177184820175 |Validation err: 0.3205, Validation loss: 0.6024591520665183
Epoch 30: Train err: 0.3085, Train loss: 0.5801699267625808 |Validation err: 0.322, Validation loss: 0.6004905047870818
Finished Training
Total time elapsed: 180.14 seconds
```

In [35]:

```
model_path = get_model_name("large", batch_size=32, learning_rate=0.001, epoch=29)
plot_training_curve(model_path)
```



Part 4. Evaluating the Best Model [15 pt]

Part (a) - 1pt

Choose the **best** model that you have so far. This means choosing the best model checkpoint, including the choice of `small_net` vs `large_net`, the `batch_size`, `learning_rate`, **and the epoch number**.

Modify the code below to load your chosen set of weights to the model object `net`.

In [37]:
```
net = LargeNet()
model_path = get_model_name(net.name, batch_size=32, learning_rate=0.001, epoch=29)
state = torch.load(model_path)
net.load_state_dict(state)
```

Out[37]:
```
<All keys matched successfully>
```

Part (b) - 2pt

Justify your choice of model from part (a).

I selected the LargeNet checkpoint from Epoch 29, trained with a batch size of 32 and a learning rate of 0.001. This configuration was the only one to achieve high accuracy without overfitting or instability. While LargeNet demonstrated better learning capacity than SmallNet, higher learning rates like 0.01 caused the validation loss to explode. Conversely, while lr=0.001 was stable, it resulted in underfitting with the default batch size of 64. By reducing the batch size to 32, I increased the update frequency, allowing the model to converge efficiently while maintaining stability. I chose the final epoch (29) because the validation loss curve was strictly decreasing right to the end, indicating the model was still generalizing well and had not yet begun to overfit.

Part (c) - 2pt

Using the code in Part 0, any code from lecture notes, or any code that you write, compute and report the **test classification error** for your chosen model.

In [38]:
```
# If you use the `evaluate` function provided in part 0, you will need to
# set batch_size > 1
# 1. Create the data loaders to ensure you have the 'test_loader' ready
train_loader, val_loader, test_loader, classes = get_data_loader(
    target_classes=["cat", "dog"],
    batch_size=64)

# 2. Define the loss function (Standard for this lab)
criterion = nn.BCEWithLogitsLoss()

# 3. Evaluate the model on the Test Set
# The evaluate function usually returns (Loss, Accuracy)
test_loss, test_acc = evaluate(net, test_loader, criterion)
```

```
# 4. Calculate and Print the Error
# Since we want Classification Error (not Accuracy), we do 1 - Accuracy
test_err = 1.0 - test_acc

print("Test Classification Error:", test_err)
```

```
Test Classification Error: 0.4051117282360792
```

Part (d) - 3pt

How does the test classification error compare with the **validation error**? Explain why you would expect the test error to be *higher* than the validation error.

In [40]:
```
print("Test Classification Error:", test_err)
# Evaluate on Validation Set
val_loss, val_acc = evaluate(net, val_loader, criterion)
val_err = 1.0 - val_acc

print("Validation Error:", val_err)
```

```
Test Classification Error: 0.4051117282360792
Validation Error: 0.40040593035519123
```

The Test Classification Error (40.5%) is slightly higher than the Validation Error (40.0%). We expect the Test Error to be higher because the Validation Set was part of the model selection process. We specifically chose our hyperparameters (Learning Rate 0.001, Batch Size 32) and our best checkpoint (Epoch 29) based on which one performed best on the validation data. This introduces a slight 'optimistic bias' toward the validation set. The Test Set, however, is completely unseen data that had zero influence on the training or selection process, providing a strictly unbiased evaluation of performance.

Part (e) - 2pt

Why did we only use the test data set at the very end? Why is it important that we use the test data as little as possible?

We only use the test data at the very end to ensure an unbiased evaluation of the model's true performance. During training, we used the validation set to tune hyperparameters (like learning rate and batch size) and select the best model checkpoint. This biases the model slightly towards the validation set. If we had used the test set for these decisions, we would be 'optimizing for the test set', making the test error an unreliable and overly optimistic estimate of how the model would perform on truly unseen real-world data.

Part (f) - 5pt

How does the your best CNN model compare with an 2-layer ANN model (no convolutional layers) on classifying cat and dog images. You can use a 2-layer ANN architecture similar to what you used in Lab 1. You should explore different hyperparameter settings to determine how well you can do on the validation dataset. Once satisified with the performance, you may test it out on the test data.

Hint: The ANN in lab 1 was applied on greyscale images. The cat and dog images are colour (RGB) and so you will need to flatted and concatinate all three colour layers before feeding them into an ANN.

In [44]:
```
import torch
import torch.nn as nn
import torch.nn.functional as F

# 1. Define the ANN Architecture
class ANN(nn.Module):
    def __init__(self):
        super(ANN, self).__init__()
        self.name = "ann"

        # Layer 1: Input (3072 features) -> Hidden Layer (30 neurons)
        # Input size = 32 pixels * 32 pixels * 3 channels = 3072
        self.fc1 = nn.Linear(32 * 32 * 3, 30)

        # Layer 2: Hidden Layer (30 neurons) -> Output (1 neuron)
        self.fc2 = nn.Linear(30, 1)

    def forward(self, x):
        # STEP 1: Flatten the image
        # Converts [Batch, 3, 32, 32] -> [Batch, 3072]
        x = x.view(-1, 32 * 32 * 3)

        # STEP 2: Feed through layers
        x = F.relu(self.fc1(x)) # Activation function
        x = self.fc2(x)         # Output layer

        # Flatten output to match label shape
        x = x.squeeze(1)
```

```
        return x

# 2. Instantiate and Train the ANN
# Note: ANNs often need slightly different hyperparameters than CNNs.
# We will start with standard defaults (Batch 64, LR 0.01)
ann_net = ANN()
train_net(ann_net, batch_size=64, learning_rate=0.01, num_epochs=30)

# 3. (Optional) Evaluate on Test Data
# Once you are happy with the validation performance above,
# you can uncomment the lines below to see the final test error.

ann_test_acc, ann_test_loss = evaluate(ann_net, test_loader, criterion)
print("ANN Test Error:", 1.0 - ann_test_acc)
```

```
Epoch 1: Train err: 0.40975, Train loss: 0.6648353652954102 |Validation err: 0.387, Validation loss: 0.6508003193885088
Epoch 2: Train err: 0.370875, Train loss: 0.6398490080833436 |Validation err: 0.402, Validation loss: 0.6601144857704639
Epoch 3: Train err: 0.360375, Train loss: 0.6287281246185302 |Validation err: 0.379, Validation loss: 0.6425877641886473
Epoch 4: Train err: 0.34625, Train loss: 0.6187239065170288 |Validation err: 0.402, Validation loss: 0.6721351183950901
Epoch 5: Train err: 0.335625, Train loss: 0.6066147575378418 |Validation err: 0.384, Validation loss: 0.6447187382727861
Epoch 6: Train err: 0.3165, Train loss: 0.5932331898212433 |Validation err: 0.366, Validation loss: 0.6450025625526905
Epoch 7: Train err: 0.313, Train loss: 0.5826071429252625 |Validation err: 0.3785, Validation loss: 0.6523102074861526
Epoch 8: Train err: 0.2925, Train loss: 0.565479304075241 |Validation err: 0.373, Validation loss: 0.6524441577494144
Epoch 9: Train err: 0.28775, Train loss: 0.5605004160404206 |Validation err: 0.384, Validation loss: 0.666878417134285
Epoch 10: Train err: 0.28025, Train loss: 0.5430123500823975 |Validation err: 0.373, Validation loss: 0.6592515241354704
Epoch 11: Train err: 0.26775, Train loss: 0.5301058158874512 |Validation err: 0.3755, Validation loss: 0.6736511886119843
Epoch 12: Train err: 0.2475, Train loss: 0.5091606738567352 |Validation err: 0.38, Validation loss: 0.712328476831317
Epoch 13: Train err: 0.246875, Train loss: 0.49537048292160035 |Validation err: 0.3745, Validation loss: 0.7141921455040574
Epoch 14: Train err: 0.237625, Train loss: 0.48975854778289796 |Validation err: 0.3855, Validation loss: 0.7402273332700133
Epoch 15: Train err: 0.225625, Train loss: 0.4721524369716644 |Validation err: 0.3665, Validation loss: 0.7642492698505521
Epoch 16: Train err: 0.21325, Train loss: 0.459338374376297 |Validation err: 0.375, Validation loss: 0.7508763149380684
Epoch 17: Train err: 0.204375, Train loss: 0.4492074680328369 |Validation err: 0.3775, Validation loss: 0.7395339962095022
Epoch 18: Train err: 0.197625, Train loss: 0.42428819370269777 |Validation err: 0.38, Validation loss: 0.7806607205420732
Epoch 19: Train err: 0.188875, Train loss: 0.41642813467979434 |Validation err: 0.3855, Validation loss: 0.8919276725500822
Epoch 20: Train err: 0.180375, Train loss: 0.4093887298107147 |Validation err: 0.3695, Validation loss: 0.8334490396082401
Epoch 21: Train err: 0.1765, Train loss: 0.3883102550506592 |Validation err: 0.3685, Validation loss: 0.8144093733280897
Epoch 22: Train err: 0.17375, Train loss: 0.39173415291309355 |Validation err: 0.3735, Validation loss: 0.877862436696887
Epoch 23: Train err: 0.14775, Train loss: 0.350450319647789 |Validation err: 0.3835, Validation loss: 0.8549375087022781
Epoch 24: Train err: 0.162, Train loss: 0.37112070429325106 |Validation err: 0.387, Validation loss: 0.914766101166606
Epoch 25: Train err: 0.142625, Train loss: 0.3353329511880875 |Validation err: 0.3755, Validation loss: 0.8731279466301203
Epoch 26: Train err: 0.13925, Train loss: 0.3287684805393219 |Validation err: 0.379, Validation loss: 1.035340290516615
Epoch 27: Train err: 0.136, Train loss: 0.33071639204025266 |Validation err: 0.38, Validation loss: 0.9781731069087982
Epoch 28: Train err: 0.12075, Train loss: 0.2952371574640274 |Validation err: 0.365, Validation loss: 0.9857132397592068
Epoch 29: Train err: 0.11875, Train loss: 0.28928155291080476 |Validation err: 0.3745, Validation loss: 1.1249216236174107
Epoch 30: Train err: 0.11, Train loss: 0.2745707542896271 |Validation err: 0.3745, Validation loss: 1.0477644745260477
Finished Training
Total time elapsed: 134.73 seconds
ANN Test Error: 0.6174999999999999
```

The huge gap between the ANN's training error (11%) and test error (61%) indicates severe overfitting. The ANN successfully memorized the training data, likely due to having a large number of parameters in the fully connected layers, but it failed completely to generalize to new images.

In [ ]:

```
# Command to convert the code to HTML for submission
%%shell
jupyter nbconvert --to html "/content/gdrive/MyDrive/Colab Notebooks/APS360/Labs/Lab2 Cats vs Dogs.ipynb"
```